

launch of RIM OS 6.x/ 7.x series with the Web kit 2 engine during late 2010-12.

4. Backward compatibility was smartly achieved by compiling a base version of the app with the minimum OS version supported by it. For all add-on delta features, add-on binary files were created for the app with a higher OS API and advanced functionality. The base code had a smart factory to check which OS version the app was running on, and dynamically loaded the implementation of the higher OS version if the device had the required OS version. While distributing the app, the distribution server would check the OS version of the device requesting the app download. For higher OS versions, the distribution server could load the JAD (Java application descriptor) file, which possessed entries of all binary files, including the base binary packaged with the minimum OS as well as add-on binary files packaged with higher OS versions to achieve advanced functionality.

## 2010: How Android and iOS transformed the mobile industry

The mobile industry underwent a disruptive transformation with the launch of Android and iOS during the years 2008-10.

Google's Android and Apple's iOS were launched to run on high-end mobile devices (like HTC, iPhone, etc). These devices were not just mobile devices but were almost small computers with great processing and computing capabilities. They had a lot of useful hardware built-in, like GPS receivers, cameras, accelerometers, etc. Android and iOS provisioned native APIs to access and use these built-in capabilities within applications.

Along with such great hardware capabilities, both Android and iOS came up with full touchscreen devices without hard keypads. This ensured that most of the screen height became usable and thus enhanced the usability of applications. Excellent gesture detection and touch capabilities of these devices provided an extraordinary user experience.

**The challenge:** J2ME, BREW and RIM were already around, and with the launch of iOS and Android, the mobile industry was using several mobile platforms between 2010 and 2012-13. Different platforms with different native APIs made it really difficult for developers/companies to build apps for mobile users.

It now became imperative for developers and companies to support all these platforms to cover the entire market. With different native APIs and different base languages, the task became really challenging. It required a different skillset to build mobile apps for different platforms, and this increased costs and time-to-market substantially, because different versions of an application were built and maintained for each platform. Domain knowledge couldn't be reused with different teams which resulted in inconsistent app behaviour.

**Web/hybrid apps:** The industry was struggling with the many variations around mobile platforms and APIs. Moreover, there were new mobile OSs/platforms, which were on the verge

of being launched. As a result, developers and companies started channelling their efforts and energies around creating platform-independent, Web/HTML based mobile applications.

Web kit engines were getting better with each new version of an OS. Also, the introduction of HTML5 gave developers the confidence to try and build mobile websites which worked across all mobile platforms. Website UIs got redesigned for single column view mobile devices and content was then served dynamically with the help of JavaScript (Ajax).

Mobile websites, a.k.a. thin clients, provided platform-independent solutions but observed less end user adoption owing to difficulties in using and managing different websites at the same time on mobile devices. Users always preferred quick app icons. Also, pure mobile websites running in browsers lacked the capability to use the inbuilt device hardware like the camera, GPS, etc.

Hybrid mobile applications overcame mobile website problems. They served Web based (HTML) content within a native mobile app container. The mobile app container houses the Web view, which loads the URL for the Web content (which can be thought of as the website's URL). Hybrid mobile apps with a quick launch app icon freed the user from managing different website URLs. Additionally, hybrid mobile apps leveraged the Web to native bridge interfaces (plugins), to utilise built-in device hardware capabilities within the app.

So hybrid mobile apps became the alternate option to deal with the time-to-market and cost problems arising from supporting multiple mobile platforms during the period 2010-12. These hybrid apps, unfortunately, soon came under scrutiny due to the adverse impact of Web content in terms of user experience and performance. This led to the famous discussion on 'native versus hybrid mobile applications'.

**Hybrid vs native mobile apps:** This has been a topic of much debate for many years amongst developers and companies wishing to create mobile applications for different platforms — whether to opt for native or hybrid mobile apps.

The best way to decide between the two options is by answering a few questions like:

1. How much does cost and time to market matter to you?
  - a. If you have a highly skilled team, and time and performance matter to you, then native is a better choice. A skilled team can quickly learn the syntax of new mobile platforms and port a native mobile app to a platform.
  - b. If your team is small and you are targeting a quick launch for your mobile app, then hybrid is the way to go, as the same code can be reused across different mobile OSs/platforms.
2. Who are your target users? Is it going to be an in-house app meant for those within the company or a public app for your customers/end consumers?
  - a. If it is going to be a public app for your customers, then native is the better choice over hybrid because the user experience is of utmost importance to mobile users. Reports on the Internet suggest that around 80 per cent